

Name - Aaryan Pratap Singh

Sap id - 590032842

Batch-15

HDOC Day-33 — Associative Properties of Matrices

1. Matrix Addition

For three matrices of the same order $(m \times n)$, verify:

$$[(A+B)+C=A+(B+C)]$$

2. Matrix Multiplication

Given:

- $(A = m \times n)$
- $(B = n \times p)$
- $(C = p \times q)$

verify:

$$[(A \times B) \times C = A \times (B \times C)]$$

Algorithm

Main Algorithm

1. Start.
2. Display the menu:
 1. Verify associative property of matrix addition.
 2. Verify associative property of matrix multiplication.
 3. Exit.
3. Read the user's choice.
4. Use **switch**:
 - Choice 1 $\rightarrow$ call **additionAssociative()**.
 - Choice 2 $\rightarrow$ call **multiplicationAssociative()**.
 - Choice 3 $\rightarrow$ terminate the program.
 - Otherwise $\rightarrow$ display "Invalid choice".
5. Repeat the menu until choice 3 is selected.
6. Stop.

Algorithm for Associative Addition

1. Read positive values of m and n .
2. Declare three VLAs $A[m][n]$, $B[m][n]$, and $C[m][n]$.
3. Read all three matrices.

4. Calculate $AB = A + B$.
5. Calculate $Left = AB + C$.
6. Calculate $BC = B + C$.
7. Calculate $Right = A + BC$.
8. Display $(A+B)+C$.
9. Display $A+(B+C)$.
10. Compare corresponding elements of $Left$ and $Right$.
11. If all elements are equal, display that matrix addition is associative.

Algorithm for Associative Multiplication

1. Read positive values m , n , p , and q .
2. Declare:
 - $A[m][n]$
 - $B[n][p]$
 - $C[p][q]$
3. Read all three matrices.
4. Calculate $AB = A \times B$, whose order is $m \times p$.
5. Calculate $Left = AB \times C$, whose order is $m \times q$.
6. Calculate $BC = B \times C$, whose order is $n \times q$.
7. Calculate $Right = A \times BC$, whose order is $m \times q$.
8. Display $(A \times B) \times C$.
9. Display $A \times (B \times C)$.
10. Compare corresponding elements.
11. If all elements are equal, display that matrix multiplication is associative.

C Program

```
#include <stdio.h>

/* Function prototypes */
void additionAssociative(void);
void multiplicationAssociative(void);

void readMatrix(int r, int c, int a[r][c]);
void printMatrix(int r, int c, int a[r][c]);

void addMatrix(int r, int c,
               int a[r][c], int b[r][c], int result[r][c]);

void multiplyMatrix(int r1, int c1, int c2,
                   int a[r1][c1],
                   int b[c1][c2],
                   int result[r1][c2]);

int compareMatrix(int r, int c,
```

```

        int a[r][c], int b[r][c]));

int main(void)
{
    int choice;

    do
    {
        printf("\n===== HDOC DAY-33 =====\n");
        printf("1. Verify associative property of matrix
addition\n");
        printf("2. Verify associative property of matrix
multiplication\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                additionAssociative();
                break;

            case 2:
                multiplicationAssociative();
                break;

            case 3:
                printf("Exiting program...\n");
                break;

            default:
                printf("Invalid choice. Please enter 1, 2 or
3.\n");
        }

        } while (choice != 3);

    return 0;
}

/* Read a matrix */
void readMatrix(int r, int c, int a[r][c])
{

```

```

    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
}

/* Display a matrix */
void printMatrix(int r, int c, int a[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            printf("%d\t", a[i][j]);
        }
        printf("\n");
    }
}

/* Add two matrices */
void addMatrix(int r, int c,
               int a[r][c], int b[r][c], int result[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            result[i][j] = a[i][j] + b[i][j];
        }
    }
}

/* Multiply two matrices */

```

```

void multiplyMatrix(int r1, int c1, int c2,
                   int a[r1][c1],
                   int b[c1][c2],
                   int result[r1][c2])
{
    int i, j, k;

    for (i = 0; i < r1; i++)
    {
        for (j = 0; j < c2; j++)
        {
            result[i][j] = 0;

            for (k = 0; k < c1; k++)
            {
                result[i][j] += a[i][k] * b[k][j];
            }
        }
    }
}

```

```

/* Compare two matrices */
int compareMatrix(int r, int c,
                  int a[r][c], int b[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            if (a[i][j] != b[i][j])
            {
                return 0;
            }
        }
    }

    return 1;
}

```

```

/* Option 1 */
void additionAssociative(void)

```

```

{
    int m, n;

    do
    {
        printf("\nEnter number of rows (m): ");
        scanf("%d", &m);

        if (m <= 0)
            printf("Rows must be positive.\n");
    } while (m <= 0);

    do
    {
        printf("Enter number of columns (n): ");
        scanf("%d", &n);

        if (n <= 0)
            printf("Columns must be positive.\n");
    } while (n <= 0);

    int A[m][n], B[m][n], C[m][n];
    int AB[m][n], BC[m][n];
    int Left[m][n], Right[m][n];

    printf("\nEnter elements of matrix A:\n");
    readMatrix(m, n, A);

    printf("\nEnter elements of matrix B:\n");
    readMatrix(m, n, B);

    printf("\nEnter elements of matrix C:\n");
    readMatrix(m, n, C);

    /* (A + B) + C */
    addMatrix(m, n, A, B, AB);
    addMatrix(m, n, AB, C, Left);

    /* A + (B + C) */
    addMatrix(m, n, B, C, BC);
    addMatrix(m, n, A, BC, Right);

    printf("\n(A + B) + C:\n");

```

```

printMatrix(m, n, Left);

printf("\nA + (B + C):\n");
printMatrix(m, n, Right);

if (compareMatrix(m, n, Left, Right))
{
    printf("\n(A + B) + C = A + (B + C)\n");
    printf("Hence, matrix addition is associative.\n");
}
else
{
    printf("\nMatrices are not equal.\n");
}
}

/* Option 2 */
void multiplicationAssociative(void)
{
    int m, n, p, q;

    do
    {
        printf("\nEnter m: ");
        scanf("%d", &m);

        if (m <= 0)
            printf("m must be positive.\n");

    } while (m <= 0);

    do
    {
        printf("Enter n: ");
        scanf("%d", &n);

        if (n <= 0)
            printf("n must be positive.\n");

    } while (n <= 0);

    do
    {
        printf("Enter p: ");

```

```

scanf("%d", &p);

if (p <= 0)
    printf("p must be positive.\n");

} while (p <= 0);

do
{
    printf("Enter q: ");
    scanf("%d", &q);

    if (q <= 0)
        printf("q must be positive.\n");

} while (q <= 0);

int A[m][n];
int B[n][p];
int C[p][q];

int AB[m][p];
int BC[n][q];

int Left[m][q];
int Right[m][q];

printf("\nEnter elements of matrix A (%d x %d):\n", m,
n);
readMatrix(m, n, A);

printf("\nEnter elements of matrix B (%d x %d):\n", n,
p);
readMatrix(n, p, B);

printf("\nEnter elements of matrix C (%d x %d):\n", p,
q);
readMatrix(p, q, C);

/* (A x B) x C */
multiplyMatrix(m, n, p, A, B, AB);
multiplyMatrix(m, p, q, AB, C, Left);

/* A x (B x C) */
multiplyMatrix(n, p, q, B, C, BC);

```

```

multiplyMatrix(m, n, q, A, BC, Right);

printf("\n(A x B) x C:\n");
printMatrix(m, q, Left);

printf("\nA x (B x C):\n");
printMatrix(m, q, Right);

if (compareMatrix(m, q, Left, Right))
{
    printf("\n(A x B) x C = A x (B x C)\n");
    printf("Hence, matrix multiplication is associative.
\n");
}
else
{
    printf("\nMatrices are not equal.\n");
}
}

```

Test Case Table

Test Case	Operation	Input	Expected Result
1	Addition	m=2, n=2, valid matrices	$(A+B)+C = A+(B+C)$
2	Addition	m=1, n=1	Both results equal
3	Addition validation	m=0	Ask for m again
4	Multiplication	m=n=p=q=2	$(A \times B) \times C = A \times (B \times C)$
5	Multiplication	m=2, n=3, p=2, q=1	Both resulting 2×1 matrices equal
6	Multiplication validation	p=-2	Ask for p again
7	Menu validation	Choice 5	Invalid choice
8	Exit	Choice 3	Program terminates

Execution

1 -

```
[aaryanrajput@aaryans-macbook ~ % nano day33.c
[aaryanrajput@aaryans-macbook ~ % clang day33.c -o day33.c
[aaryanrajput@aaryans-macbook ~ % ./day33.c

===== HDOC DAY-33 =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 1

Enter number of rows (m): 2
Enter number of columns (n): 2

Enter elements of matrix A:
1 2 3 4

Enter elements of matrix B:
5 6 7 8

Enter elements of matrix C:
9 10 11 12

(A + B) + C:
15      18
21      24

A + (B + C):
15      18
21      24

(A + B) + C = A + (B + C)
Hence, matrix addition is associative.
```

2-

```
===== HDOC DAY-33 =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 2

Enter m: 2
Enter n: 2
Enter p: 2
Enter q: 2

Enter elements of matrix A (2 x 2):
1 2 3 4

Enter elements of matrix B (2 x 2):
5 6 7 8

Enter elements of matrix C (2 x 2):
1 0 0 1

(A x B) x C:
19      22
43      50

A x (B x C):
19      22
43      50

(A x B) x C = A x (B x C)
Hence, matrix multiplication is associative.

===== HDOC DAY-33 =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: █
```